

Trabajo Práctico Especial:

Primera Entrega

Autoría:

Manuel Ader #63587 – mader@itba.edu.ar

Materia:

(2024-2Q) 72.39 - Autómatas,
Teoría de Lenguajes y Compiladores

Profesores:

Arias Roig, Ana Maria

Golmar, Mario Agustin

Fecha de entrega: 24/11/2024

Índice

Índice.....	2
1. Introducción.....	3
2. Modelo Computacional.....	4
2.1. Dominio.....	4
2.2. Lenguaje.....	5
3. Implementación.....	6
3.1. Frontend.....	6
3.2. Backend.....	8
3.4. Dificultades Encontradas.....	9
4. Futuras Extensiones.....	11
5. Conclusiones.....	11
6. Referencias.....	11

C-- :

Un Lenguaje de Propósito General

1. Introducción

C-- es un lenguaje de programación de propósito general (GPL) diseñado para ser imperativo y Turing completo. El objetivo principal fue crear un lenguaje que pudiera traducirse a C mientras mantiene una sintaxis más simple y accesible. El lenguaje soporta las funcionalidades básicas necesarias para desarrollar programas de complejidad arbitraria.

Una característica distintiva de C-- es su manejo de variables globales, que a diferencia de C, todas las variables tienen alcance global en el programa, lo que significa que no se puede declarar dos variables con el mismo nombre independientemente del contexto (por eso el nombre C--).

2. Modelo Computacional

2.1. Dominio

El lenguaje implementa tres tipos de datos básicos:

INT: Para valores numéricos enteros

BOOL: Para valores booleanos

STRING: Para cadenas de caracteres

Las operaciones soportadas incluyen:

Operaciones aritméticas: +, -, , / y agrupación con paréntesis

Operaciones lógicas: AND, OR, NOT

Operaciones de comparación: ==, >, >=, <, <=

Las estructuras de control implementadas son:

Declaraciones de variables

Asignaciones

Funciones con parámetros y retorno

Condicionales (IF-ELSE)

Bucles (FOR)

Impresión (PRINT)

2.2. Lenguaje

La sintaxis del lenguaje es simple y está inspirada en C, aunque está diseñada para ser más intuitiva aun. Aquí hay algunos ejemplos representativos:

Operaciones aritméticas básicas:

```
PROGRAM {  
  
    INT x = 3 + 2;  
  
    PRINT(x);  
}
```

```
}
```

Funciones:

```
PROGRAM {  
    INT suma(INT a, INT b) {  
        RETURN a + b;  
    }  
    RETURN suma(3, 4);  
}
```

Ciclos:

```
PROGRAM {  
    INT sum = 0;  
    FOR i IN (1, 5) {  
        sum = sum + i;  
    }  
    PRINT(sum);  
}
```

Condicionales:

```
PROGRAM {  
    INT a = 5;  
    INT b = 3;
```

```
IF(a > b) {  
    PRINT(a);  
} ELSE {  
    PRINT(b);  
}  
  
RETURN 0;  
}
```

3. Implementación

3.1. Frontend

El análisis léxico y sintáctico se implementó utilizando Flex y Bison. La gramática está diseñada para realizar validaciones de tipos directamente en las reglas gramaticales, evitando la necesidad de un sistema de typechecking separado.

La tabla de símbolos se implementó como una linkedlist, permitiendo un manejo eficiente de las declaraciones y referencias de variables y funciones.

Las validaciones semánticas se encuentran en BisonActions.c, particularmente en las funciones que involucran tipos de datos, por ejemplo AssignmentSemanticAction():

```
"switch (expression->type) {  
  
    case ARITHMETIC_EXPR_T:  
  
        if (varType->type != INT_T) typeError = true;
```

```
        break;

    case BOOLEAN_EXPR_T:

        if (varType->type != BOOL_T) typeError = true;

        break;

    case STRING_EXPR_T:

        if (varType->type != STRING_T) typeError = true;

        break;

}

if (typeError) {

    logError(_logger, "Error: Type mismatch in assignment to '%s'",
varName);

}"
```

En este fragmento de código se valida que si estamos asignando una variable a una expresión aritmética, que la misma variable sea de tipo INT, lo mismo para el caso de expresión booleana y de tipo STRING.

3.2. Backend

La generación de código C se realiza mediante la traducción directa del AbstractSyntaxTree. El proceso se divide en dos fases principales:

Generación de declaraciones de funciones

Generación de la función main con el resto de las instrucciones

La implementación de la generación de código se encuentra en Generator.c. a continuación un ejemplo de generación de código:

```
“
static void generateConditional(Conditional * conditional, int
indentLevel) {
    _output(indentLevel, "if (");
    generateBooleanExpression(conditional->condition);
    _output(0, ") {\n");

    generateBlock(conditional->ifBlock, indentLevel + 1);

    if (conditional->elseBlock != NULL) {
        _output(indentLevel, "} else {\n");
        generateBlock(conditional->elseBlock, indentLevel + 1);
    }

    _output(indentLevel, "}\n");
}”
```

3.4. Dificultades Encontradas

Los principales desafíos técnicos incluyeron:

- Diferenciación entre nombres de variables y funciones:

Se resolvió implementando una expresión regular con lookahead para detectar el paréntesis de apertura en las llamadas a funciones.

Solución:

```
[a-zA-Z_][a-zA-Z0-9_]*      { return VariableIdentifierLexemeAction
(createLexicalAnalyzerContext()); }
```

```
[a-zA-Z_][a-zA-Z0-9_]*/"(" { return FunctionIdentifierLexemeAction
(createLexicalAnalyzerContext()); }
```

- Manejo de múltiples tokens a partir de una misma expresión regular:

Se implementó utilizando una variable global que almacena el último tipo declarado para decidir qué tipo de token enviar.

Solución:

```
Type* identifierType = NULL;

Symbol* symbol = findSymbol(identifier);

if (symbol != NULL) {

// If the identifier already exists in the symbol table use its type

if (symbol->kind == VARIABLE) identifierType = symbol->type;

} else {

// If the identifier does not exist in the symbol table, a new
symbol is created

if (_type != NULL) { // _type is global variable

    identifierType = createType(_type->type);

    addSymbol(identifier, VARIABLE, identifierType);

}

}

if (identifierType != NULL) {
```

```
switch (identifierType->type) {  
    case INT_T:  
        return INT_VAR_NAME;  
    case BOOL_T:  
        return BOOL_VAR_NAME;  
    case STRING_T:  
        return STRING_VAR_NAME;  
}  
}
```

4. Futuras Extensiones

Posibles mejoras para el lenguaje incluyen:

- Implementación de alcance local para variables
 - Adición de más tipos de datos
 - Soporte para arrays y estructuras de datos
 - Optimizaciones en el código generado
-

5. Conclusiones

El desarrollo de C-- cumplió con los objetivos planteados de crear un lenguaje de propósito general traducible a C. El proyecto permitió comprender en profundidad la implementación práctica de un compilador utilizando Flex y

Bison, así como los desafíos que implica el diseño de un lenguaje de programación.

6. Referencias

Documentación de Flex y Bison

Especificación del lenguaje C

Apuntes propios de clases teóricas y prácticas de la materia.
